

PSC - GPU Computing & MPI

All experiments in this report were ran using the slurm scripts included within the submission.

A1. Vector Triad

The vector kernels were evaluated using problem sizes $N \in \{5 \times 10^6, 2.5 \times 10^7, 5 \times 10^7\}$. Since each value is stored within a single precision float, a vector of length 5×10^7 occupies approximately 200MB of memory. The COPY kernel touches two vectors, $\mathbf{x}$ and $\mathbf{y}$, reading from $\mathbf{x}$ and writing to $\mathbf{y}$, therefore using around 400MB of memory. The AXPY kernel also touches two vectors, reading $\mathbf{x}$ and reading/writing to $\mathbf{y}$, so uses roughly 400MB of memory. The ADD kernel uses three vectors, reading from both $\mathbf{x}$ and $\mathbf{y}$, and writing to $\mathbf{z}$, so uses about 600MB of memory. The largest cases therefore, are far larger than the cache found on chip, so place the kernels in a regime where the dominant cost is global memory traffic.

The global vector of size N is distributed equally across all MPI ranks, so each rank operates locally on a vector of size $N_{\text{local}} = \frac{N}{\text{Number of Ranks}}$. Each rank launches CUDA kernels on its local data.

Each of the kernels supports arbitrary values of N , even when it is not a multiple of the CUDA block size. This is achieved through the use of grid-stride loops. In a grid-stride loop, each thread begins at index i , where $i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$. And while $i < N$, it advances by $\text{blockDim.x} \times \text{gridDim.x}$. This makes it so all elements can be processed without being padded or aligned to the block size.

The performance of the kernels was reported using effective memory bandwidth in GB/s. Where it was calculated by dividing the total global memory traffic by kernel runtime:

$$\text{GB/s} = \frac{\text{bytes moved}}{10^9 \times \text{seconds}},$$

where the kernel runtime was measured with CUDA event timing around the kernel launch, so reported values represent the device-side performance of the kernel and don't show device transfer times.

The read/write accounting used for each kernel was as follows:

For AXPY, each element performs one read from $\mathbf{x}$, one read from $\mathbf{y}$, and one write back to $\mathbf{y}$, giving a total of 3 memory accesses per element:

$$\text{bytes moved} = 3 \times N_{\text{local}} \times \text{sizeof(float)} = 12 \times N_{\text{local}}$$

For ADD, each element performs one read from $\mathbf{x}$, one read from $\mathbf{y}$, and one write to $\mathbf{z}$, again also giving:

$$\text{bytes moved} = 3 \times N_{\text{local}} \times \text{sizeof(float)} = 12 \times N_{\text{local}}$$

For COPY, each element performs one read from $\mathbf{x}$ and one write to $\mathbf{y}$, giving:

$$\text{bytes moved} = 2 \times N_{\text{local}} \times \text{sizeof(float)} = 8 \times N_{\text{local}}$$

Kernel	Problem Size	Runtime (ms)	Effective Memory Bandwidth (GB/s)
AXPY	5×10^6	0.050752	1182.22
	2.5×10^7	0.194944	1538.90
	5×10^7	0.379456	1581.21
ADD	5×10^6	0.043008	1395.09
	2.5×10^7	0.194560	1541.94
	5×10^7	0.389120	1541.94
COPY	5×10^6	0.046976	851.499
	2.5×10^7	0.152832	1308.63
	5×10^7	0.285888	1399.15

Table 1: Kernel performance across varying problem sizes, measured with 1 MPI rank.

A2. Parallel Reduction

At the start of each parallel reduction kernel, an initial index is assigned to each thread, based on block and thread identifiers. Per iteration, threads can load up to two input elements, $\mathbf{x}[i]$ and $\mathbf{x}[i + \text{blockDim.x}]$, this is done to improve memory throughput by increasing the work done by each thread. A grid strided loop is then

used so that kernels can process arrays larger than the number of threads launched. During this phase, threads accumulate assigned values into a private register variable `sum`, done to minimise writes to shared and global memory.

After all of a thread’s elements have been accumulated, the thread stores its local sum in shared memory. The block then carries out a tree based reduction in this shared memory, halving the number of active values each step. After each shared memory update, it is ensured that all writes are visible before the next reduction step by using a thread synchronisation barrier.

When the number of values has been reduced to the point of one warp, the implementation switches to a warp level reduction using CUDA shuffle instructions. `__shfl_down_sync` is used to exchange values directly between threads in the same warp, this allows completion of the final reduction with no further shared memory accesses or synchronisation across the entire block. Improving efficiency because threads within a warp execute in lockstep, so explicit synchronisation is not needed. This results in lane 0 of the warp holding the sum for the entire block, which is then written to the output array of partial sums.

The full reduction is completed with multiple kernel launches. The first stage, `k_reduce_block_sum` reduces the original input array into one partial sum for each block. Later stages, `k_reduce_partials` use the same reduction strategy for the partial sum array from the previous stage. This repeats until there is one partial sum left. Multiple kernels are used because standard CUDA kernels don’t give global synchronisation across every block with a single launch.

To validate the accuracy of the reduction kernel, it was tested against a CPU benchmark for different sized N for 10 different seeds across MPI ranks $\{1, 2, 4\}$. Absolute and relative errors were measured across these runs, and compared against tolerances. Tolerance was based on IEEE-754 single precision floating point accuracy [1]. As the GPU performs a parallel float reduction while the CPU reference uses a double precision sequential accumulation, exact agreement is not expected. So, the relative tolerance was chosen as a small multiple of the single precision machine epsilon, while the absolute tolerance was scaled with problem size to account for the accumulation of rounding errors and to stay meaningful when the true sum is close to zero.

N	MPI Ranks	Avg Abs Error	Avg Rel Error	Abs Tolerance	Rel Tolerance
10000000	1	0.000155640	1.72764e-07	3.16228	1.19209e-05
	2	0.000250244	6.26818e-07		
	4	0.000195313	1.66920e-07		
30882137	1	0.000309753	5.09548e-07	5.55717	1.19209e-05
	2	0.000296021	2.08067e-07		
	4	0.000250244	1.14325e-07		
50000000	1	0.000435257	1.41726e-06	7.07107	1.19209e-05
	2	0.000537109	2.05713e-07		
	4	0.000341797	5.96714e-08		

Table 2: Validation results for the GPU reduction against the CPU reference across multiple input sizes and MPI rank counts.

B1. Naive GPU GEMM

The naive GEMM implementation has one GPU thread per output element $C(i, j)$. Within each thread, the global row and column indices are computed from the CUDA block and thread indices. If (i, j) falls within the matrix bounds, the thread does a dot product between row i of A and column j of B , accumulating over $k = 0, \dots, K - 1$. All of the operands are read from global memory inside this loop and the final value is written to $C(i, j)$.

B2. Tiled GEMM using Shared Memory

In the optimised tiled GEMM kernel 2D tiling and shared memory are used. Every thread block produces one tile of the output matrix C , while subtiles of A and B are read from global memory and stored in shared memory arrays. After each tile is loaded, the threads synchronise with each other using `__syncthreads()` before carrying out partial multiply accumulate operations for that tile. Then, another synchronisation is used before the next tile is loaded in the K dimension. When loading tiles and writing the final result, boundary checks are used, this allows for the implementation to handle matrix sizes that are not multiples of the tile size.

Three different tile sizes were tested: 8×8 , 16×16 , and 32×32 across different problem sizes with $M = N = K$.

For these tests, the performance metric used was GFLOP/s, where:

$$\text{GFLOP/s} = \frac{2 \times M_{\text{local}} \times N \times K}{10^9 \times \text{computation time(s)}}$$

The results for this can be seen in table 3:

Tile Size	Problem Size	Computation Time (ms)	Communication Time (ms)	GFLOP/s
8×8	2048	7.21203	3.67038	2382.11
	4096	55.6227	13.5326	2470.92
	8192	455.742	54.3450	2412.57
16×16	2048	4.85888	3.74801	3535.77
	4096	36.8343	13.8635	3731.28
	8192	300.132	53.5036	3663.42
32×32	2048	4.38682	3.70089	3916.25
	4096	34.3480	14.1439	4001.36
	8192	276.284	53.7945	3979.64

Table 3: Performance of different GEMM tile sizes across different problem sizes, with 1 MPI rank.

The 32×32 version achieved the best performance across the problems, having the lowest total runtime for each tested scenario. Therefore, the 32×32 tiled kernel is used for the main performance comparison.

Kernel	M	N	K	Computation time (ms)	GFLOP/s
Naive	4096	4096	4096	57.6113	2385.63
Tiled	4096	4096	4096	34.3245	4004.11

Table 4: Performance comparison of naive and shared-memory tiled GPU GEMM for $M = N = K = 4096$ using 1 MPI rank.

Table 4 shows that the shared memory tiled implementation outperforms the naive GPU kernel for the same problem size. The tiled kernel reduces computation time from 57.6113 ms to 34.3245 ms, giving a speedup of $1.68\times$ and increasing performance from 2385.63 GFLOP/s to 4004.11 GFLOP/s.

This occurs because in the naive kernel, each thread computes one output element and reads all the needed values from **A** and **B** from global memory in each inner loop iteration. Because of this, there are repeated memory accesses for data from global memory, that are reused by multiple threads, so performance is heavily limited by the memory bandwidth and latency. Whereas, in the tiled kernel, subblocks of **A** and **B** are loaded into shared memory using 32×32 tiles. After synchronisation, each loaded value can be reused by multiple threads within the thread block before the next tile is fetched. The reuse of this data lowers the global memory traffic and so improves effective arithmetic intensity, leading to higher GFLOP/s in the tiled kernel.

C1. Strong Scaling Experiment for Matrix Multiplication

For this experiment, the total GPU compute times were measured, along with communication times, comprising of individual times for scatter, broadcast and barriers.

MPI ranks	Total (ms)	Comp. (ms)	Comm. (ms)	Scatter (ms)	Bcast (ms)	Barrier (ms)
1	48.5058	34.3562	14.1496	14.1390	0.0090	0.0016
2	102.3236	77.0836	25.2400	25.2262	0.0115	0.0023
4	97.1805	62.1261	35.0544	35.0446	0.0082	0.0016

Table 5: Strong scaling results for tiled GPU GEMM (32×32 tile) with fixed global problem size $M = N = K = 4096$. Reported values for multi-rank runs correspond to the slowest rank.

Because this experiment was carried out on a single GPU, runtimes increased with the number of MPI ranks. As a result of this, adding more ranks did not give extra GPU compute resources, rather, it resulted in several MPI processes sharing the same device while also facing increased communication and synchronisation costs.

As a result of this, the experiment does not show distributed strong scaling across multiple GPUs, but instead shows the overhead of increasing ranks with a single GPU available. The compute times do not decrease significantly from 1 to 4 ranks, as the same GPU is still performing all of the work, while overheads from communication grow quickly. Specifically the scatter time becomes the dominant cost at larger ranks, causing

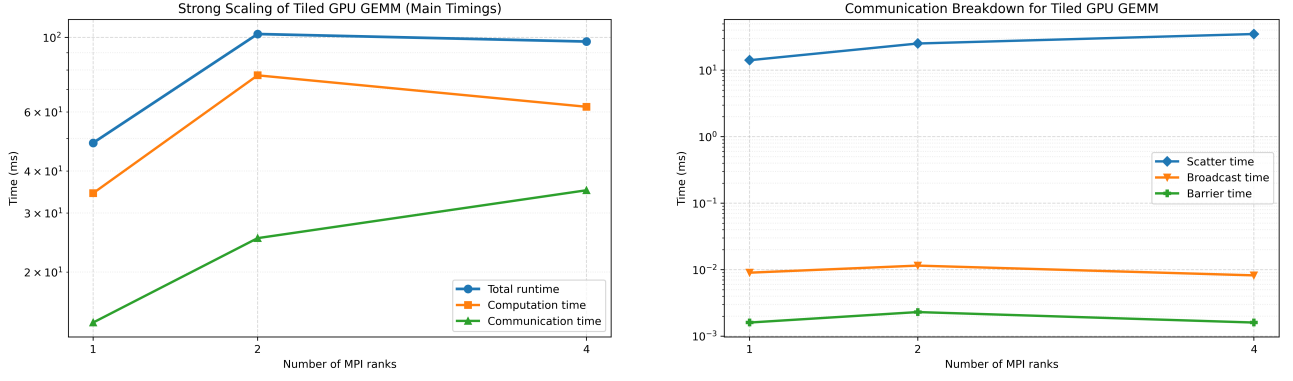


Figure 1: MPI scaling results on a log scale. Left: main result. Right: communication breakdown. Number of ranks vs. Time.

large increases in the total runtime. This shows that distributing data across the ranks adds large costs, while any potential benefits are hampered by ranks contending for the same accelerator.

The results show that MPI communication outweighs GPU computation when multiple ranks are ran on the same GPU. In a system where multiple GPUs are available, each rank would ideally have access to individual GPUs, allowing the local computation for each rank to decrease and for there to be parallel execution. In this experiment, because there was only a single GPU, the costs of overhead and resource contention are shown, rather than the scalability of the GEMM kernel.

C2. Profiling & Bottleneck Analysis

Nsight systems was used to profile a tiled GEMM run with 4 MPI ranks at a fixed problem size $M = N = K = 4096$ on a single GPU. The profile showed that the tiled kernel was executed 4 times, with a total GPU kernel time of approximately 195.5ms and an average per rank kernel duration of 48.9ms. CUDA memory transfers made up around 207.4ms for host to device copies and 81.6ms for device to host copies. The overhead of kernel launches was very small with a total `cudaLaunchKernel` time of 4.10ms.

The profile shows that the kernel launch frequency is low and its launching overheads are not a significant bottleneck for the GEMM implementation. The largest costs come from communication and waiting on the host side. The measured communication time for the 4 rank run came in at 32-38ms for each rank, of which scatter made up the largest component, while the broadcasting and barrier times were negligible. Showing that the overheads from distributing data across MPI ranks were the main bottleneck to performance at a high number of MPI ranks. The overall runtime is larger than the total GPU kernel time, this shows that the limiting resource for the application is not the GPU, but rather the increased communication and coordination overheads that come with increasing MPI ranks without adding more GPU compute capacity.

If more time were available, the next optimisations I would implement would be to reduce communication costs and were possible try to overlap communication with computation. In a setting with multiple GPUs, assigning a GPU to each individual MPI rank would remove the contention on the a shared device and allow for a scaling report that shows more meaningful results. Other improvements that could be made could be lowering the need to broadcast matrix B, through the use of asynchronous communication, and looking into if it would be possible to pipeline communication and data transfers with kernel execution.

References

- [1] Sun Microsystems, Inc., *Numerical Computation Guide*, Sun Microsystems, Inc., Palo Alto, CA, May 2000, revision A, Part No. 806-3568-10.